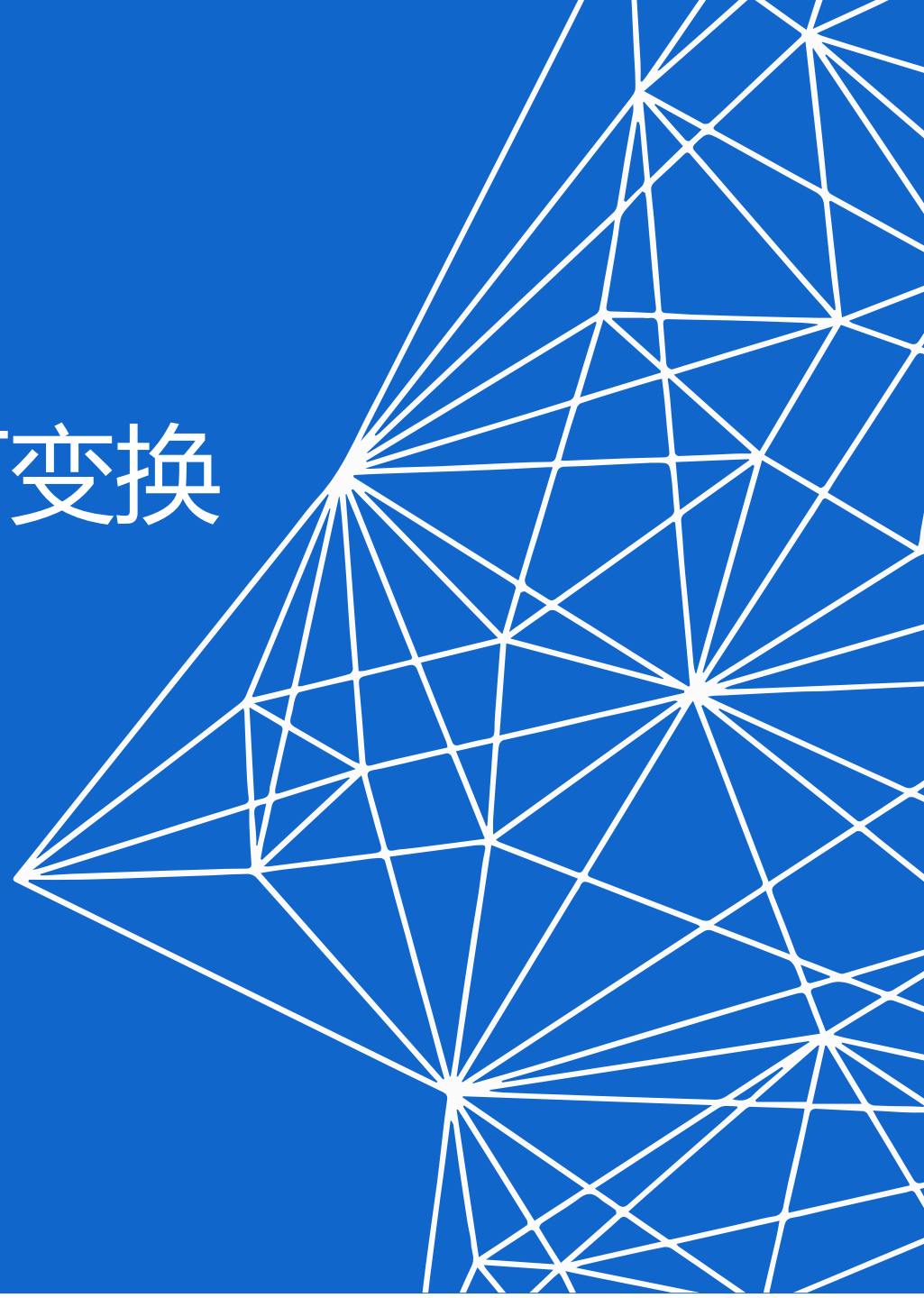




第四节：图像缩放与几何变换

杨聪

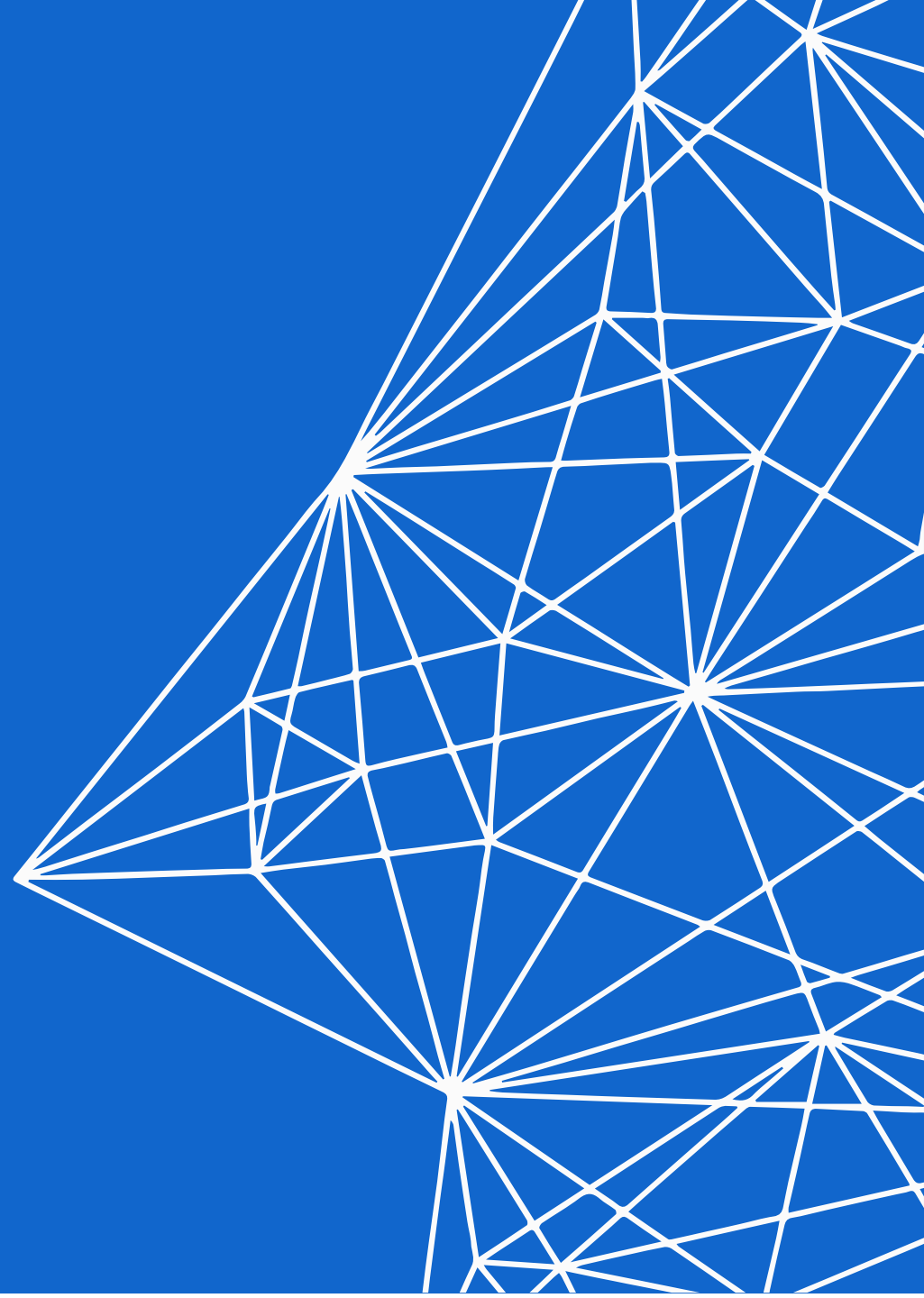




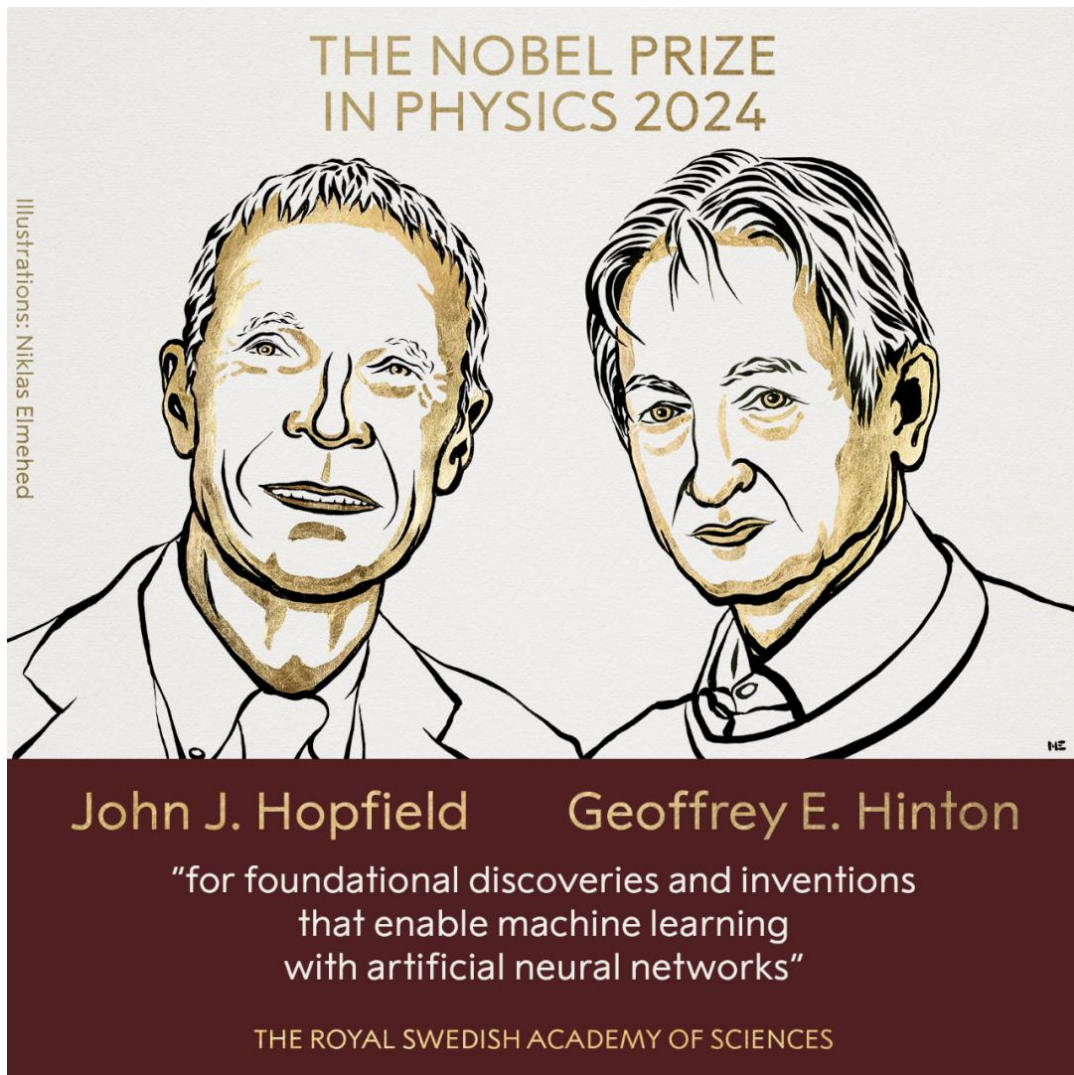
目录

CONTENTS

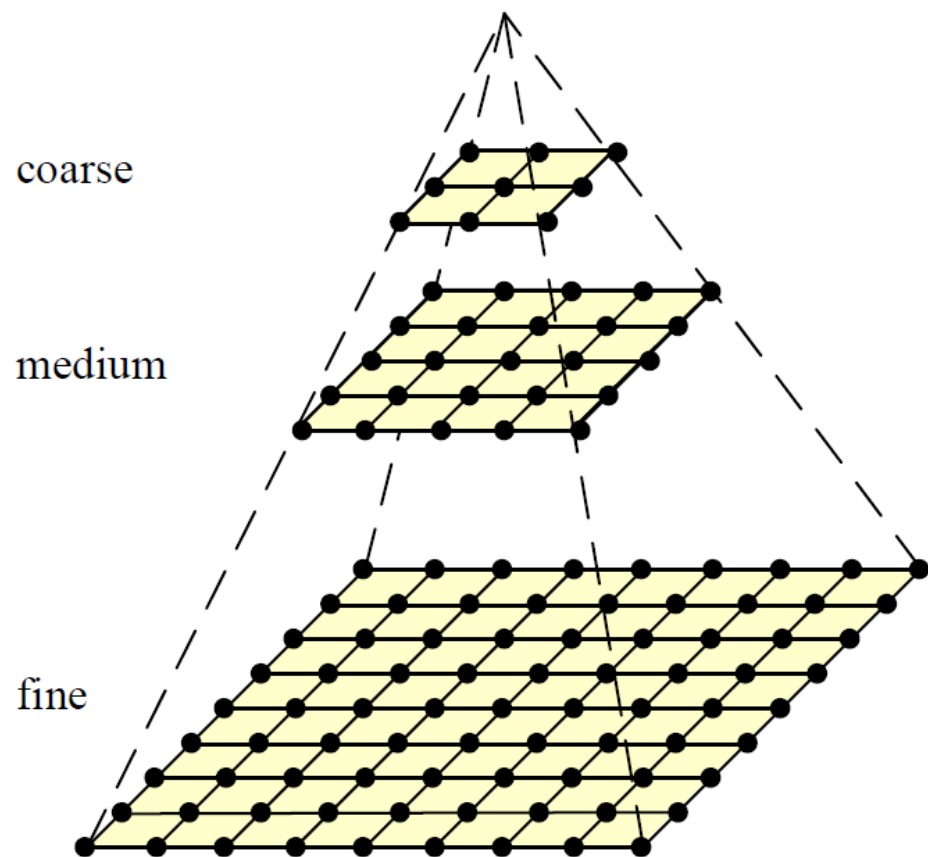
1. 图像金字塔
 - 插值
 - 多分辨率表达
2. 图像几何变换



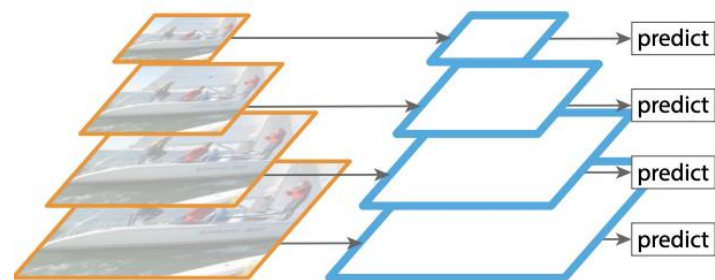
诺贝尔奖



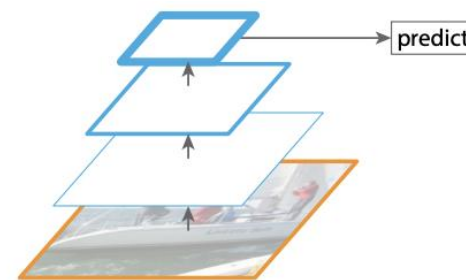
图像金字塔



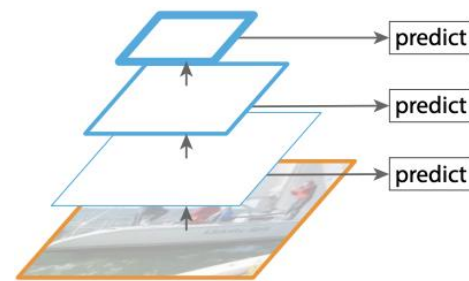
图像金字塔模型



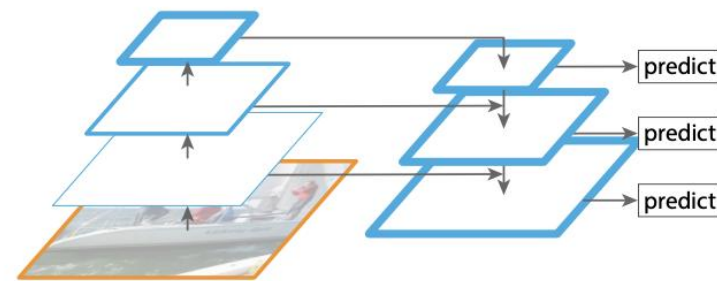
(a) Featurized image pyramid



(b) Single feature map



(c) Pyramidal feature hierarchy

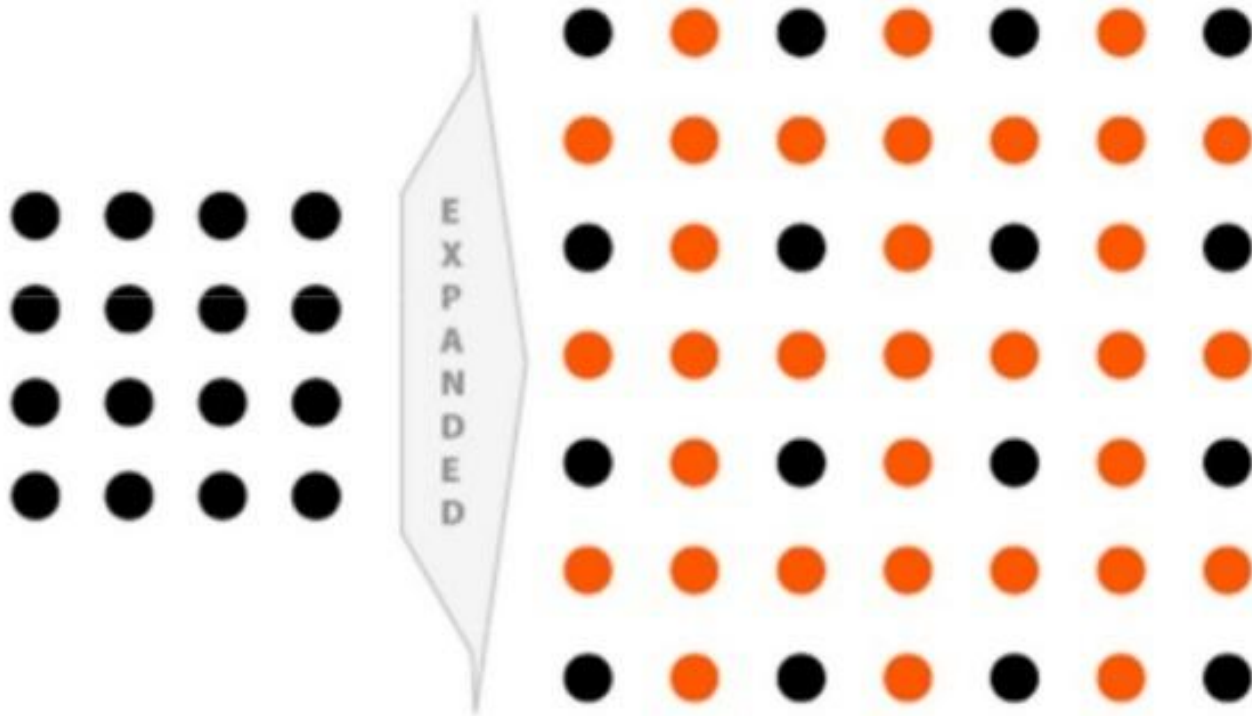


(d) Feature Pyramid Network

深度学习金字塔模型

图像插值

在图像几何变换时，无法给有些像素点直接赋值，例如，将图像放大两倍，必然会多出一些无法被直接映射的像素点，对于这些像素点，通过插值决定它们的值。于是，产生了图像插值算法。

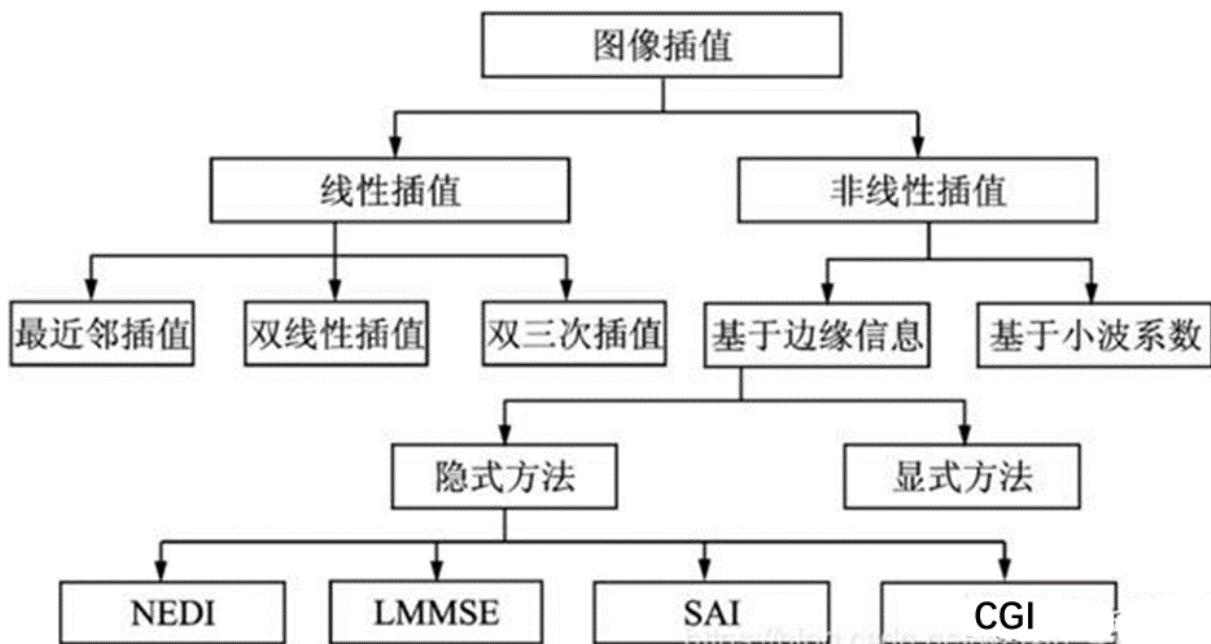


图像插值

主要可以分为两类，一类是线性图像插值方法，另一类是非线性图像插值方法

传统的插值方法如最近邻插值，双线性插值以及双三次插值等都属于线性插值方法。这类插值方法在图像插值过程中采用同一种插值内核，不用考虑待插像素点所处的位置，这种做法会使图像中的边缘变得模糊不清，达不到高清图像的视觉效果。

非线性插值方法主要包括：基于小波系数的方法基于边缘信息的方法。其中，基于边缘信息的方法又可分为隐式方法和显式方法两种。隐式方法包含：边缘导向插值(New edge directive interpolation, NEDI)，最小均方误差估计插值(Linear minimum mean square-error estimation, LMMSE)、软判决自适应插值(Soft-decision adaptive interpolation interpolation, SAI)，边缘对比度引导的图像插值(Contrast-guideed image interpolation, CGI)等都是隐式的基于图像边缘的插值方法。显式方法如Hermite曲线插值。

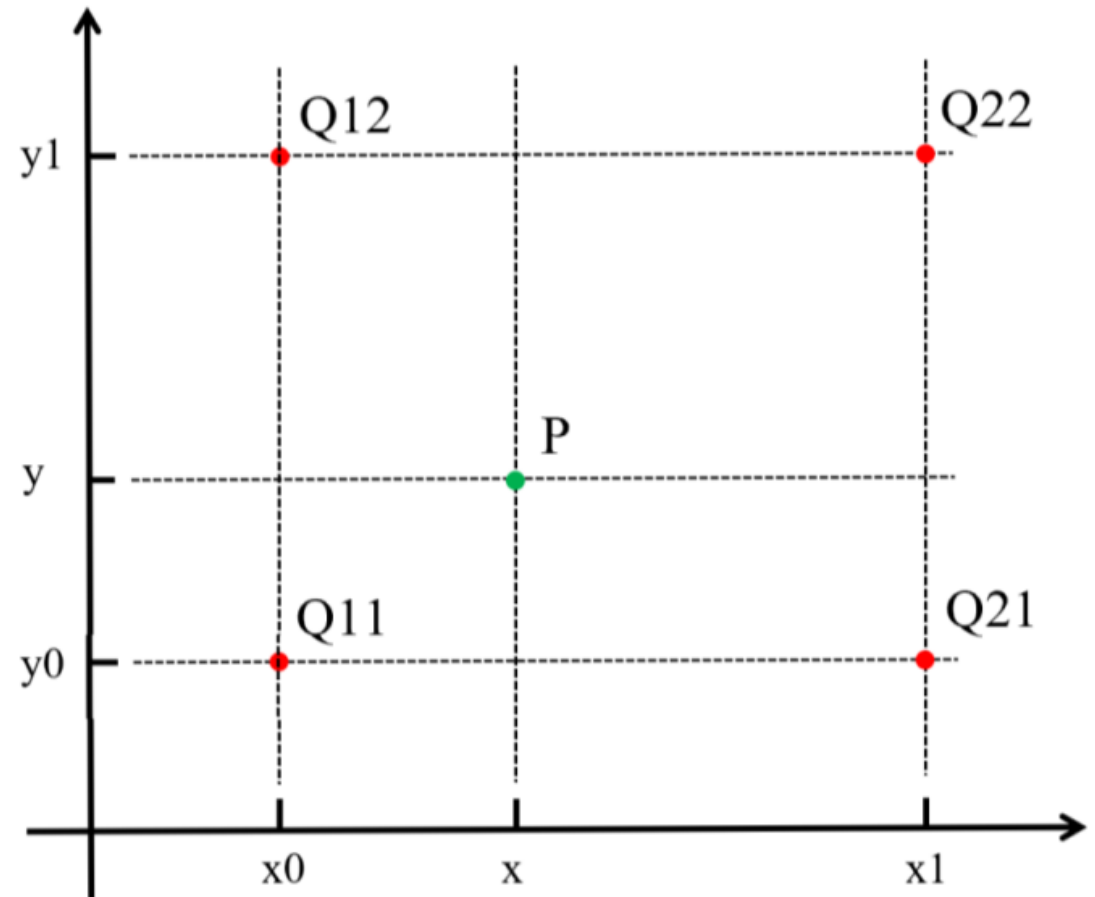


最新发展的话有基于决策树，深度学习，字典学习的图像插值算法。

最近邻插值算法

原理：将目标图像中的点，对应到原图像中后，找到最相邻的整数坐标点的像素值，作为该点的像素值输出。

如右图所示，目标图像中的某点投影到原图像中的位置为点P，与P距离最近的点为Q11，此时易知， $f(P)=f(Q11)$ 。

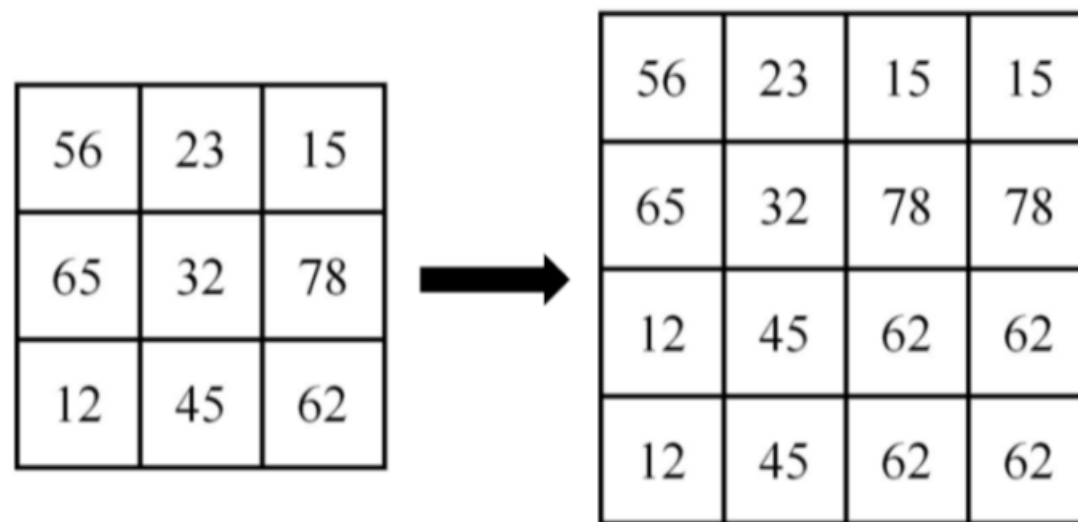


最近邻插值算法

将一幅3*3图像放大到4*4，用 $f(x, y)$ 表示原图像， $h(x, y)$ 表示目标图像，我们有如下公式：

$$h(dstX, dstY) = f(\frac{dstX * src_width}{dst_width}, \frac{dstY * src_height}{dst_height})$$

$$\begin{aligned} h(0, 0) &= f(0, 0) \\ h(0, 1) &= f(0, 0.75) = f(0, 0) \\ h(0, 2) &= f(0, 1.50) = f(0, 1) \\ h(0, 3) &= f(0, 2.25) = f(0, 2) \\ &\dots \end{aligned}$$



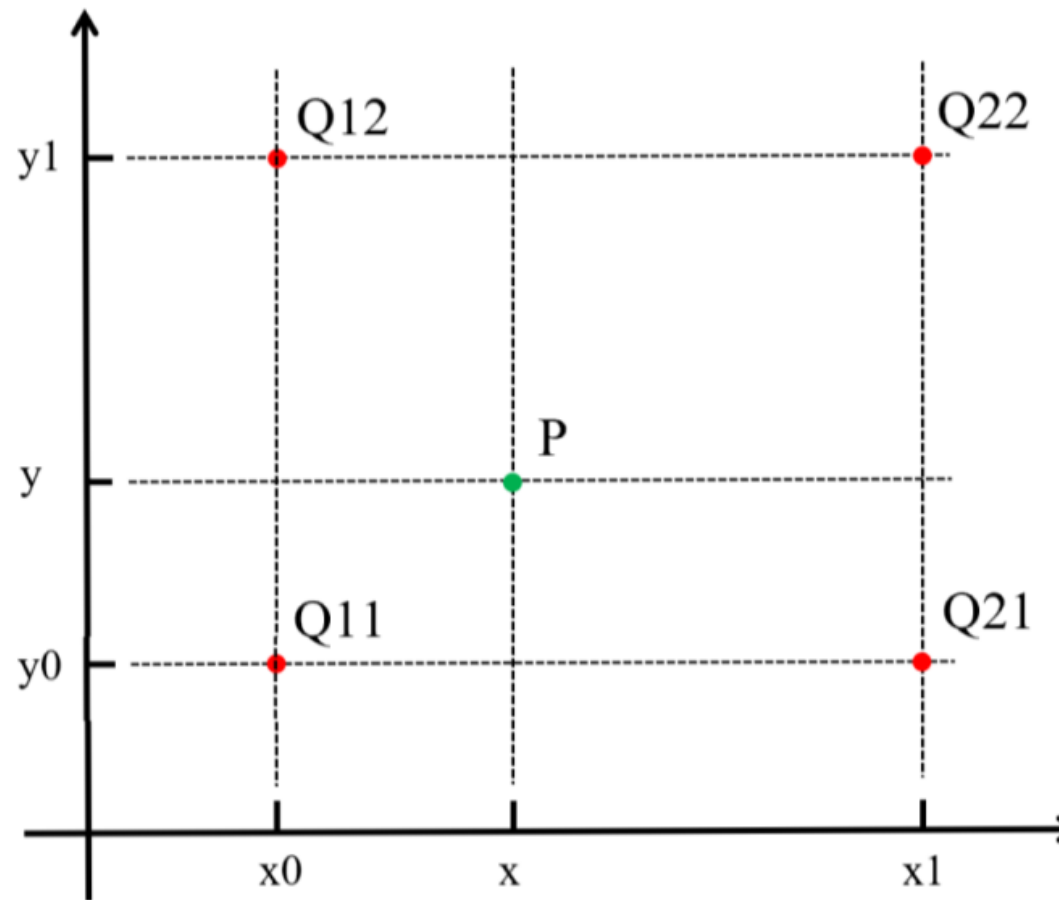
最近邻插值算法

缺点：

放大后的图像有很严重的马赛克，会出现明显的块状效应；缩小后的图像有很严重的失真。

这是一种最基本、最简单的图像缩放方式。变换后的每个像素点的像素值，只由原图像中的一个像素点确定。例如上面，点 $(0,0.75)$ 的像素只由 $(0,1)$ 确定，这样的效果显然不好。点 $(0,0.75)$ 的像素不止和 $(0,1)$ 有关，和 $(0,0)$ 也有关，只是 $(0,1)$ 的影响更大。

如果可以用附近的几个像素点按权重分配，共同确定目标图像某点的像素，效果会更好。下面的双线性插值就解决了这个问题。



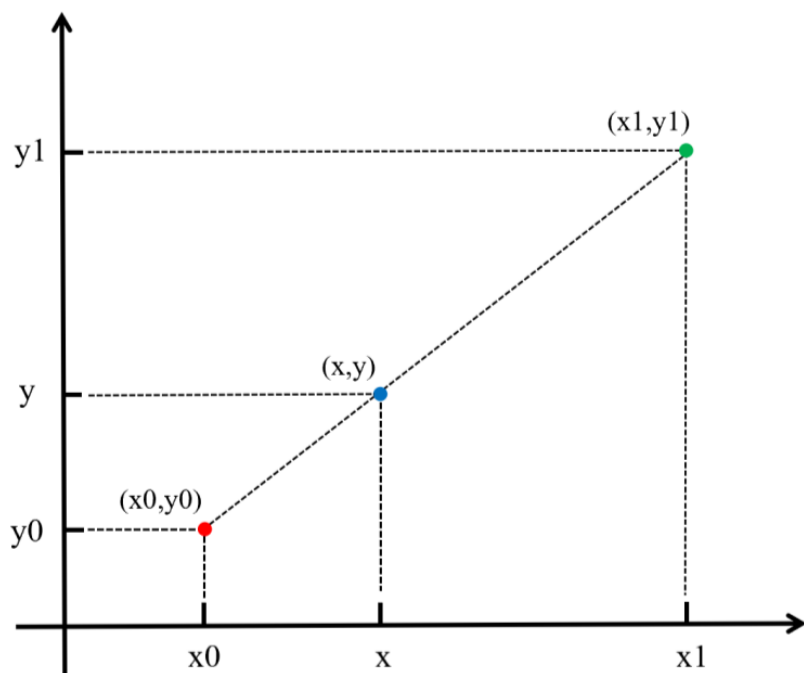


双线性插值算法

在讲双线性插值之前先了解一下线性插值。

线性插值：使用连接两个已知量的直线来确定在这两个已知量之间的一个未知量的值。线性插值形式：

$$f(x) = a_1x + a_0$$



线性插值多项式：

$$y = y_0 + (x - x_0) \frac{y_1 - y_0}{x_1 - x_0} = y_0 + \frac{(x - x_0) y_1 - (x - x_0) y_0}{x_1 - x_0}$$

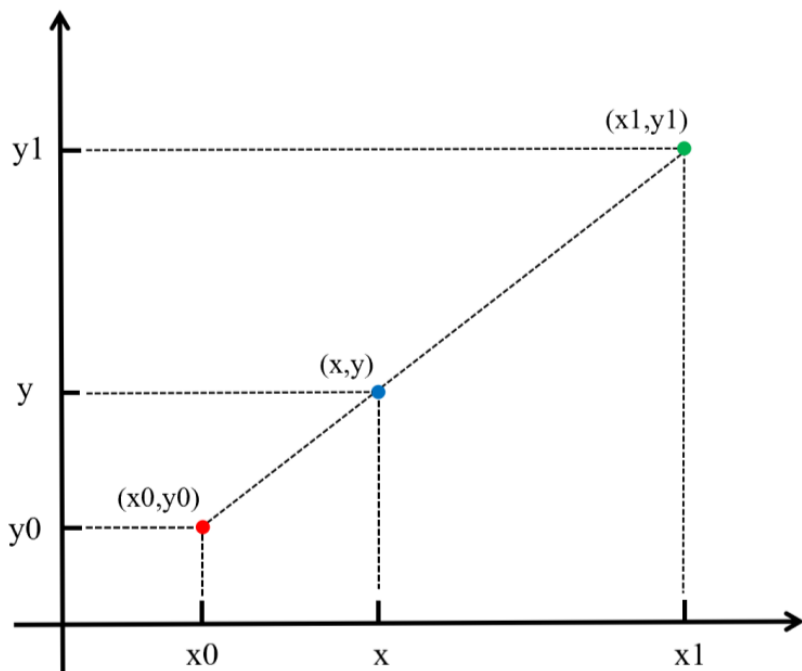
其实，即使 x 不在 x_0 到 x_1 之间，这个公式也是成立的。在这种情况下，这种方法叫作线性外插。

双线性插值算法

在讲双线性插值之前先了解一下线性插值。

线性插值：使用连接两个已知量的直线来确定在这两个已知量之间的一个未知量的值。线性插值形式：

$$f(x) = a_1x + a_0$$



线性插值多项式：

$$y = y_0 + (x - x_0) \frac{y_1 - y_0}{x_1 - x_0} = y_0 + \frac{(x - x_0)y_1 - (x - x_0)y_0}{x_1 - x_0}$$

线性插值的误差：线性插值其实就是拉格朗日插值有2个结点时的情况。插值余项为：

$$R_n(x) = \frac{f''(\xi)(x - x_0)(x - x_1)}{2!} \leq \frac{f''(\xi)(x_0 - x_1)^2}{8}, \xi \in (x_0, x_1)$$

双线性插值算法

在讲双线性插值之前先了解一下线性插值。

线性插值：使用连接两个已知量的直线来确定在这两个已知量之间的一个未知量的值。线性插值形式：

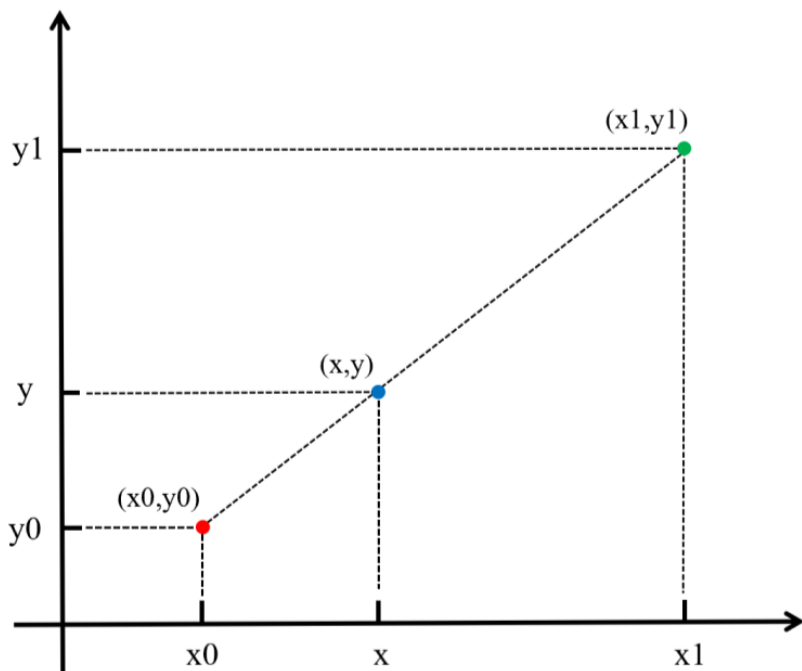
$$f(x) = a_1x + a_0$$

线性插值多项式：

$$y = y_0 + (x - x_0) \frac{y_1 - y_0}{x_1 - x_0} = y_0 + \frac{(x - x_0)y_1 - (x - x_0)y_0}{x_1 - x_0}$$

线性插值的误差：线性插值其实就是拉格朗日插值有2个结点时的情况。插值余项为：

$$R_n(x) = \frac{f''(\xi)(x - x_0)(x - x_1)}{2!} \leq \frac{f''(\xi)(x_0 - x_1)^2}{8}, \xi \in (x_0, x_1)$$



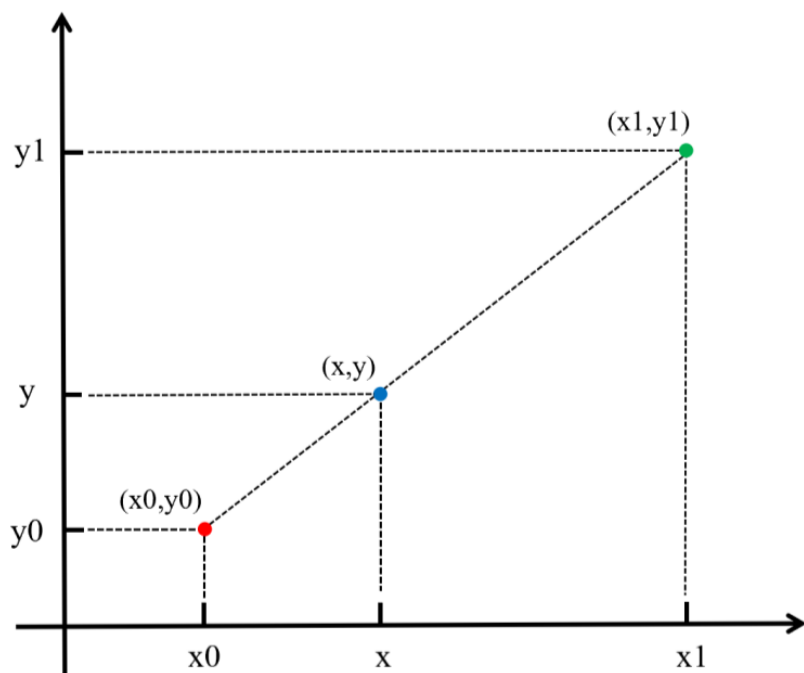
从插值余项可以看出，随着二阶导数的增大，线性插值的误差增大。即函数的曲率越大，线性插值近似的误差也越大。

双线性插值算法

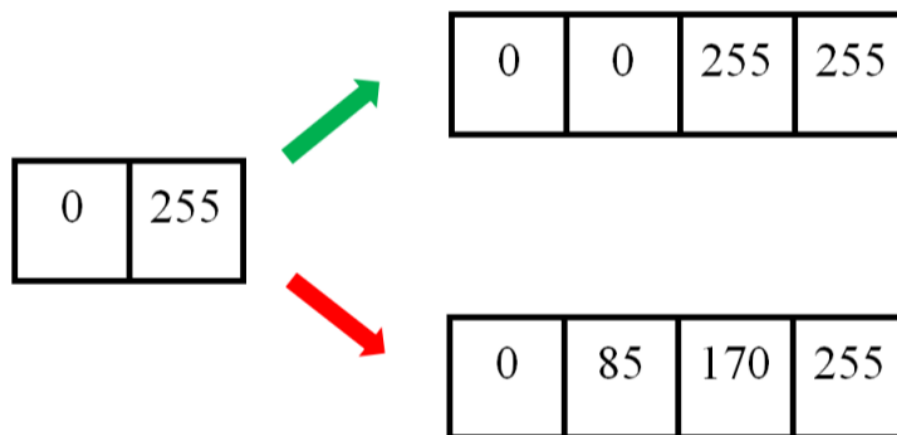
在讲双线性插值之前先了解一下线性插值。

线性插值：使用连接两个已知量的直线来确定在这两个已知量之间的一个未知量的值。线性插值形式：

$$f(x) = a_1x + a_0$$



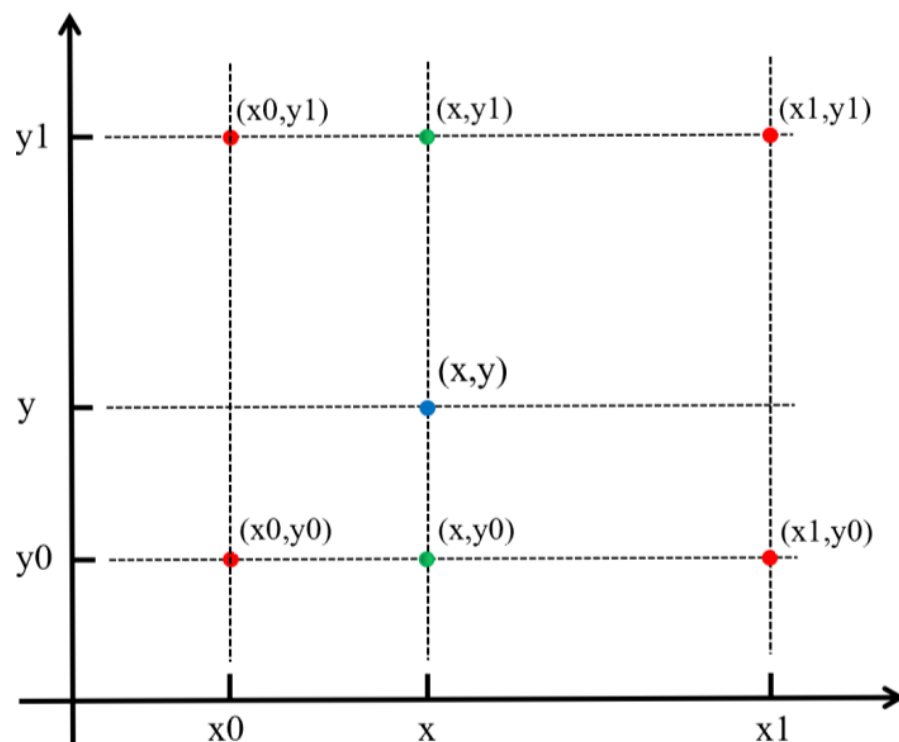
举个例子：左边为原图像，拉伸后，理想的输出图像的像素分布应该为绿色箭头指向的，但是按照线性插值，会得到红色箭头指向的结果。



双线性插值算法

双线性插值形式: $f(x, y) = ax + by + cxy + d$

双线性插值是线性插值在二维时的推广,在两个方向上共做了三次线性插值。具体操作为在X方向上进行两次线性插值计算,然后在Y方向上进行一次插值计算。如下图所示:



首先, $f(x, y)$ 为二元函数, 假设我们知道 $f(x_0, y_0), f(x_1, y_1), f(x_0, y_1), f(x_1, y_0)$ 四个点的值。这四个点确定一个矩形, 我们希望通过插值得到矩形内任意点的函数值。

先在x方向上进行两次线性插值, 得到:

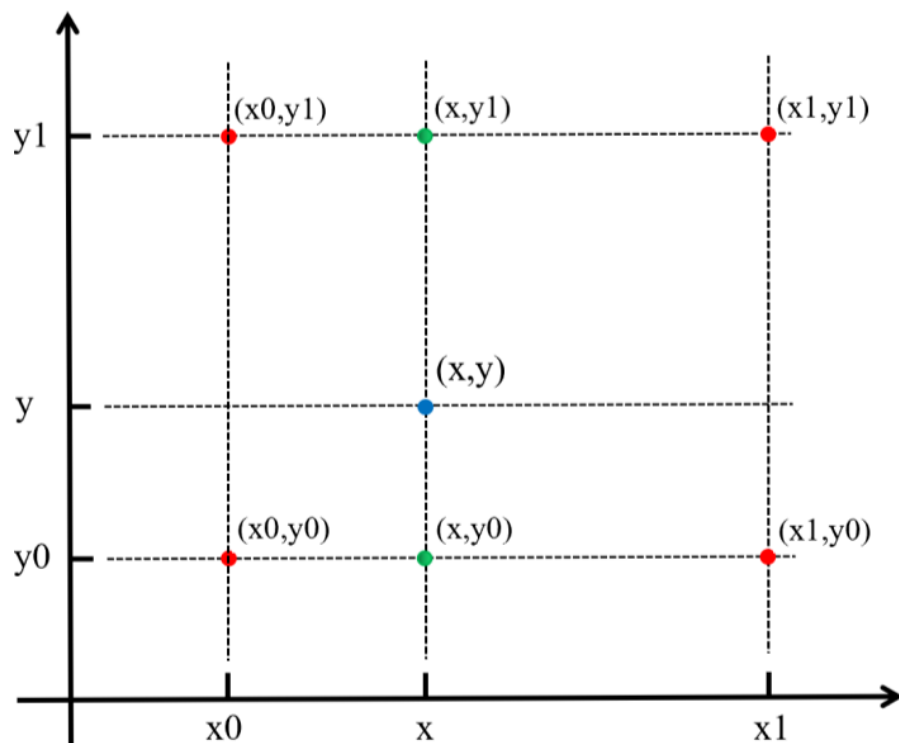
$$f(x, y_0) = \frac{x_1 - x}{x_1 - x_0} f(x_0, y_0) + \frac{x - x_0}{x_1 - x_0} f(x_1, y_0)$$

$$f(x, y_1) = \frac{x_1 - x}{x_1 - x_0} f(x_0, y_1) + \frac{x - x_0}{x_1 - x_0} f(x_1, y_1)$$

双线性插值算法

双线性插值形式: $f(x, y) = ax + by + cxy + d$

双线性插值是线性插值在二维时的推广,在两个方向上共做了三次线性插值。具体操作为在X方向上进行两次线性插值计算,然后在Y方向上进行一次插值计算。如下图所示:



先在x方向上进行两次线性插值, 得到:

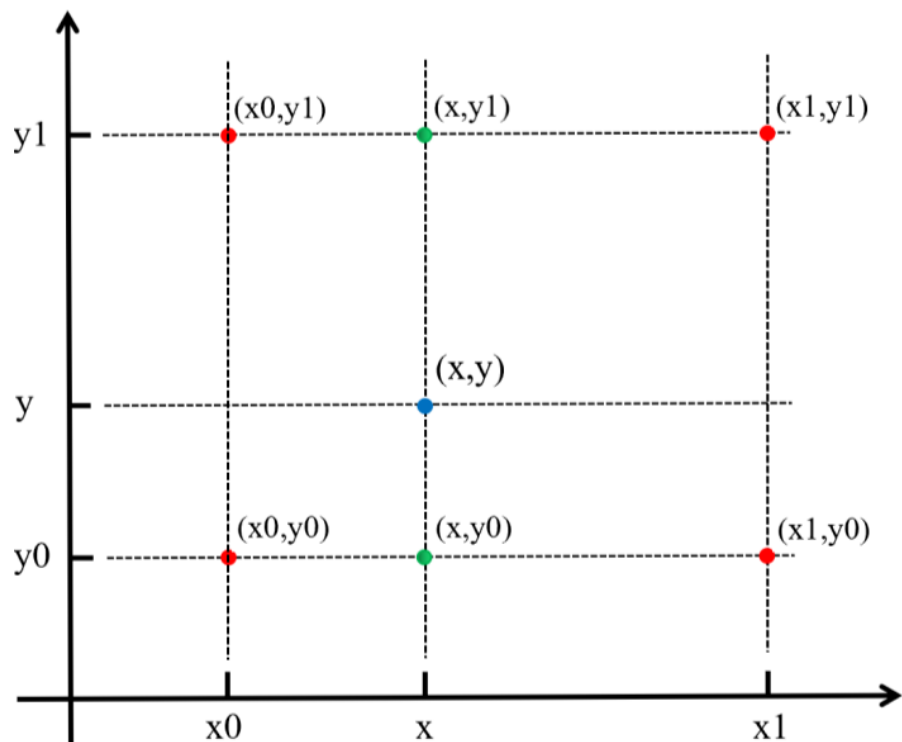
$$f(x, y_0) = \frac{x_1 - x}{x_1 - x_0} f(x_0, y_0) + \frac{x - x_0}{x_1 - x_0} f(x_1, y_0)$$

$$f(x, y_1) = \frac{x_1 - x}{x_1 - x_0} f(x_0, y_1) + \frac{x - x_0}{x_1 - x_0} f(x_1, y_1)$$

再在y方向上进行一次线性插值, 得到:

$$f(x, y) = \frac{y_1 - y}{y_1 - y_0} f(x, y_0) + \frac{y - y_0}{y_1 - y_0} f(x, y_1)$$

双线性插值算法



先在x方向上进行两次线性插值，得到：

$$f(x, y_0) = \frac{x_1 - x}{x_1 - x_0} f(x_0, y_0) + \frac{x - x_0}{x_1 - x_0} f(x_1, y_0)$$

$$f(x, y_1) = \frac{x_1 - x}{x_1 - x_0} f(x_0, y_1) + \frac{x - x_0}{x_1 - x_0} f(x_1, y_1)$$

再在y方向上进行一次线性插值，得到：

$$f(x, y) = \frac{y_1 - y}{y_1 - y_0} f(x, y_0) + \frac{y - y_0}{y_1 - y_0} f(x, y_1)$$

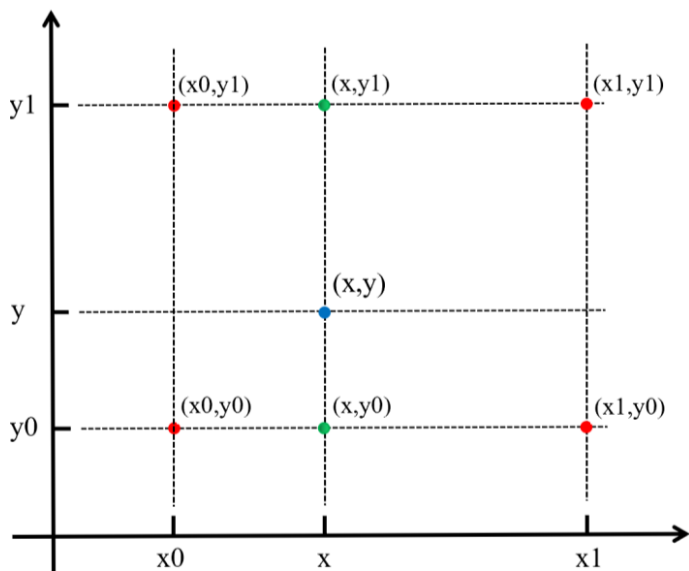
综合起来，就是双线性插值的结果：

$$f(x, y) = \frac{(y_1 - y)(x_1 - x)}{(y_1 - y_0)(x_1 - x_0)} f(x_0, y_0) + \frac{(y_1 - y)(x - x_0)}{(y_1 - y_0)(x_1 - x_0)} f(x_1, y_0) + \frac{(y - y_0)(x_1 - x)}{(y_1 - y_0)(x_1 - x_0)} f(x_0, y_1) + \frac{(y - y_0)(x - x_0)}{(y_1 - y_0)(x_1 - x_0)} f(x_1, y_1)$$



双线性插值算法

如果选择一个坐标系，使 $f(x)$ 已知的四个点的坐标分别为 $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$ ，那么确定一个单位正方形，四个点分别为正方形的四个顶点：



首先对上端的两个顶点进行线性插值得：

$$f(x, 0) = f(0, 0) + x[f(1, 0) - f(0, 0)]$$

再对底端的两个顶点进行线性插值得：

$$f(x, 1) = f(0, 1) + x[f(1, 1) - f(0, 1)]$$

最后，做垂直方向的线性插值，以确定：

$$f(x, y) = f(x, 0) + y[f(x, 1) - f(x, 0)]$$

整理得插值公式的化简形式：

$$f(x, y) = [f(1, 0) - f(0, 0)]x + [f(0, 1) - f(0, 0)]y + [f(1, 1) + f(0, 0) - f(0, 1) - f(1, 0)]xy + f(0, 0)$$

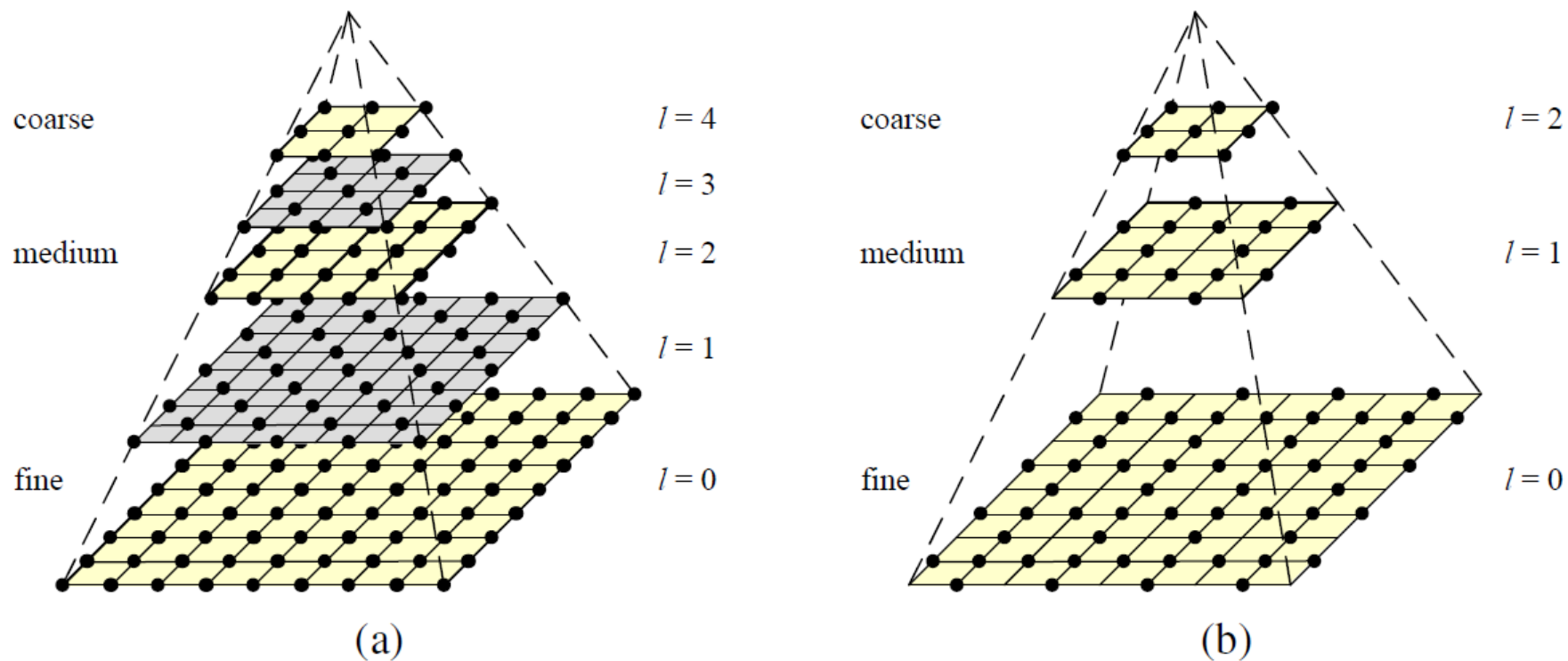


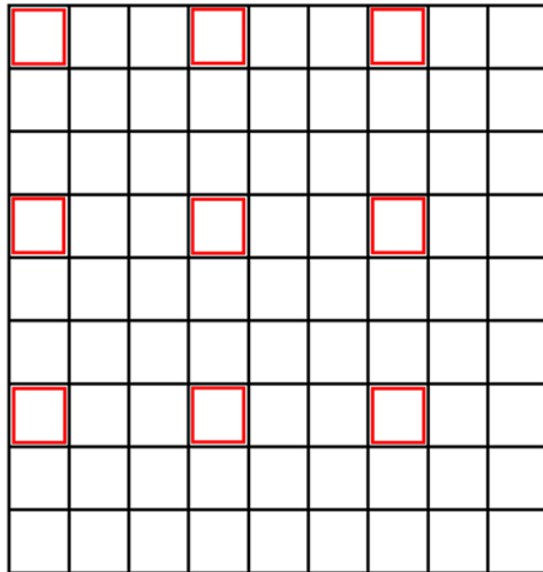
Figure 3.35 Multiresolution pyramids: (a) pyramid with half-octave (quincunx) sampling (odd levels are colored gray for clarity). (b) wavelet pyramid—each wavelet level stores $3/4$ of the original pixels (usually the horizontal, vertical, and mixed gradients), so that the total number of wavelet coefficients and original pixels is the same.

原图像和目标图像的几何中心对齐

在计算目标图像中，对应原图像的虚拟坐标点时，一般的变换是：

$$\begin{aligned}srcX &= dstX (src_{width} / dst_{width}) \\srcY &= dstY (src_{height} / dst_{height})\end{aligned}$$

这种变换下，原图像的有些点没有参与计算。



举个例子：

从图片可以看出，只有圈出来的红色部分参与运算了。

目标图像的每个像素点的灰度值相对于原图像偏左上方，右下角的元素实际上没有参与运算。



原图像和目标图像的几何中心对齐

在计算目标图像中，对应原图像的虚拟坐标点时，一般的变换是：

$$\begin{aligned}srcX &= dstX(src_width / dst_width) \\srcY &= dstY(src_height / dst_height)\end{aligned}$$

为了让原图像和目标图像的中心对齐，我们规定另外一种变换方式：

$$\begin{aligned}srcX &= dstX(src_width / dst_width) + 0.5(src_width / dst_width - 1) \\srcY &= dstY(src_height / dst_height) + 0.5(src_height / dst_height - 1)\end{aligned}$$

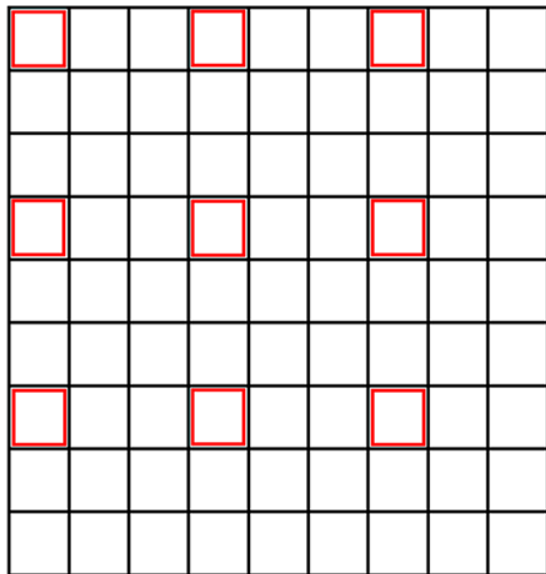
就是在原来的变换后面加了调节因子： $0.5(src_width / dst_width - 1)$

- 放大图像时， $src_width / dst_width > 1$ ，就加了一个正数，相当于向右移动了。
- 缩小图像时， $src_width / dst_width < 1$ ，就加了一个负数，相当于向左移动了。

原图像和目标图像的几何中心对齐

为了让原图像和目标图像的中心对齐，我们规定另外一种变换方式：

$$\begin{aligned}srcX &= dstX(src_width / dst_width) + 0.5(src_width / dst_width - 1) \\srcY &= dstY(src_height / dst_height) + 0.5(src_height / dst_height - 1)\end{aligned}$$



这种变换下，

目标图像的中心点(1,1)，对应了原图像的中心点(4,4)，两个图像的几何中心重合，能充分利用原图像的点，并且目标图像的每个像素点之间都是等间隔的，也都和两边有一定的边距。

实际上，在openCv中也是这种变换方式



cv2.resize()的计算过程

对于缩小图像，目标图像中每个点都能找到原图像中包围它的四个临近点，每个点都进行双线性插值即可。

对于放大图像，边界附近的点经过坐标变换可能超出了原图像的范围。举个例子，把 3×3 的原图像放大成 4×4 。

- 中间的点：双线性插值

中间的点都能在原图像中找到包围它的四个临近点，做双线性插值即可。

- 边界上的点（除了顶点）：线性插值

例如，目标图像中的点 $(1,3)$ ，对应原图像的点为 $(0.625,2.125)$ ，原图像的纵坐标最大为2，找不到包围 $(0.625,2.125)$ 四个点，所以用它最邻近的两个点 $(0,2)$ 和 $(1,2)$ 做线性插值（外插），得到目标图像中 $(1,3)$ 的像素值。

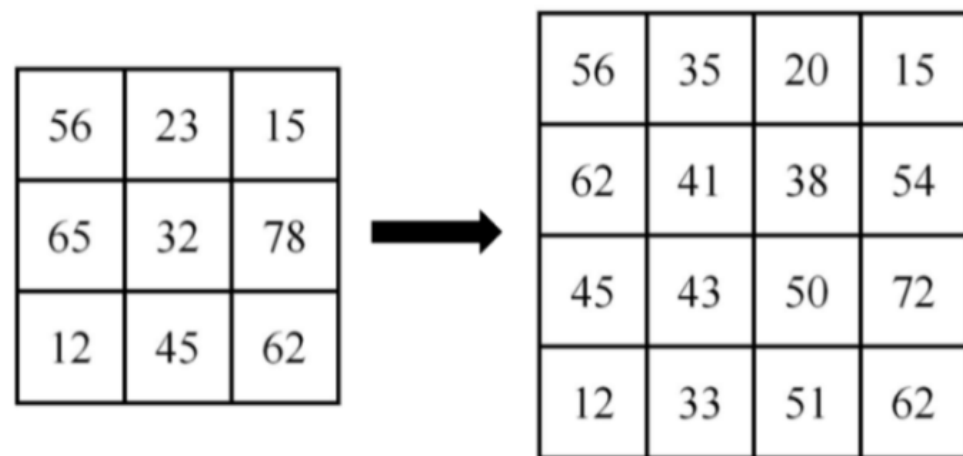
- 四个顶点：最邻近插值

例如，目标图像右上角的顶点 $(0,3)$ ，对于原图像的点为 $(0,2.125)$ ，直接用原图像右上角的顶点 $(0,2)$ 作为它的值即可。

多分辨率表达

cv.resize()的计算过程

用 $h(x,y)$ 表示目标图像, $f(x,y)$ 表示原图像



中间的点: 双线性插值

$$\begin{aligned}
 h(1,1) &= f(0.625, 0.625) \\
 &= [f(1,0) - f(0,0)] * 0.625 + [f(0,1) - f(0,0)] * 0.625 + [f(1,1) + f(0,0) - f(0,1) - f(1,0)] * 0.625^2 + f(0,0) \\
 &= (23 - 56) * 0.625 + (65 - 56) * 0.625 + [32 + 56 - 23 - 65] * 0.625^2 + 56 \\
 &= 41 \\
 &\dots\dots
 \end{aligned}$$

也可以算出 $f(1,2)$, $f(2,1)$ 和 $f(2,2)$

多分辨率表达

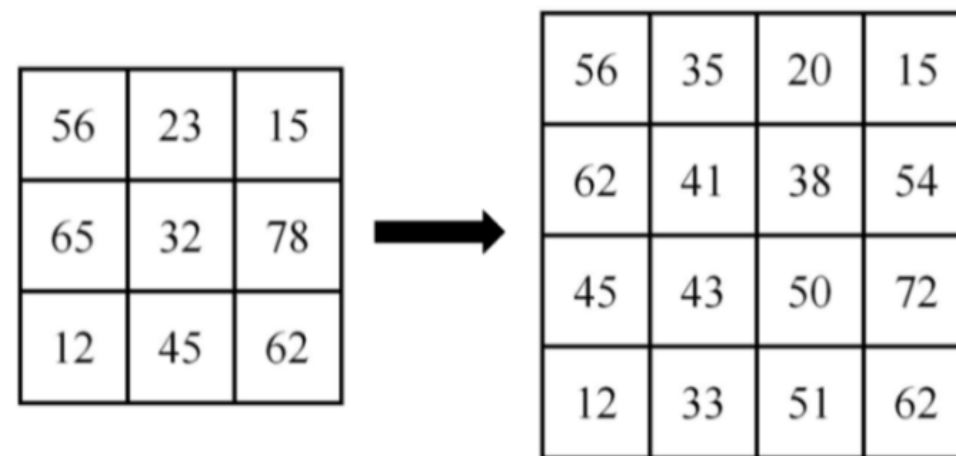
cv.resize()的计算过程

用 $h(x,y)$ 表示目标图像, $f(x,y)$ 表示原图像

边界上的点 (除了顶点): 线性插值

$$\begin{aligned} h(0,1) &= f(-0,125,0.625) \\ &= f(0,0) + 0.625 * [f(0,1) - f(0,0)] \\ &= 56 - 20.625 \\ &= 35(\text{四舍五入取整}) \end{aligned}$$

$$\begin{aligned} h(0,2) &= f(-0,125,1.375) \\ &= f(0,1) + (1.375 - 1) * [f(0,2) - f(0,1)] \\ &= 23 - 3 \\ &= 20 \\ &\dots \end{aligned}$$



多分辨率表达

cv2.resize()的计算过程

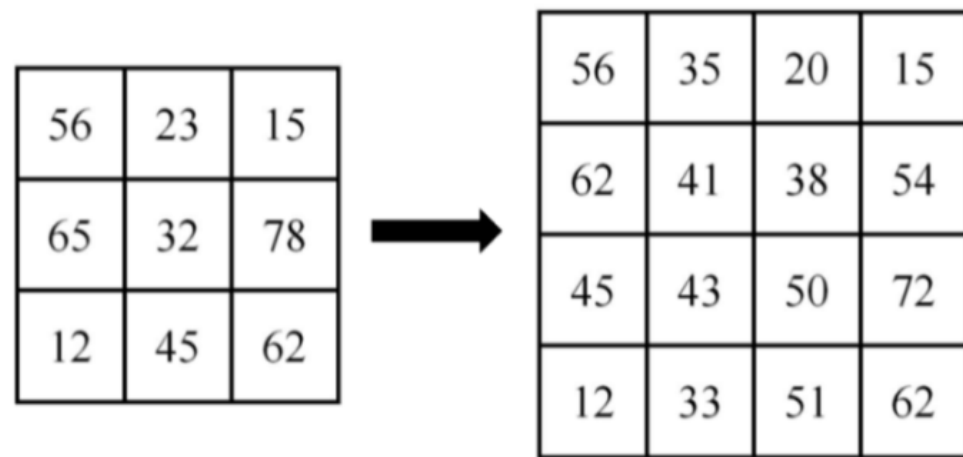
用 $h(x,y)$ 表示目标图像, $f(x,y)$ 表示原图像

四个顶点: 最邻近插值

$$h(0,0) = f(-0.125, -0.125) = f(0,0) = 56$$

$$h(3,3) = f(2.125, 2.125) = f(2,2) = 62$$

.....





多分辨率表达

#dst为输出图像，类型与原图像相同

```
dst = cv2.resize(src, dsize[, dst[, fx[, fy[, interpolation]]]])
```

参数	描述
src	【必需】原图像
dsize	【必需】输出图像的大小
fx	【可选】沿水平轴的比例因子
fy	【可选】沿垂直轴的比例因子
interpolation	【可选】插值方式

类型	说明
cv.INTER_NEAREST	最近邻插值
cv.INTER_LINEAR	双线性插值（默认方式）
cv.INTER_CUBIC	基于4x4像素邻域的3次插值法
cv.INTER_AREA	区域插值，基于局部像素的重采样

多分辨率表达



原图像



0.2倍缩小
双线性插值



缩小后的图像1.5倍放大
最近邻插值



缩小后的图像1.5倍放大
双线性插值



原图像



0.3倍缩小
双线性插值

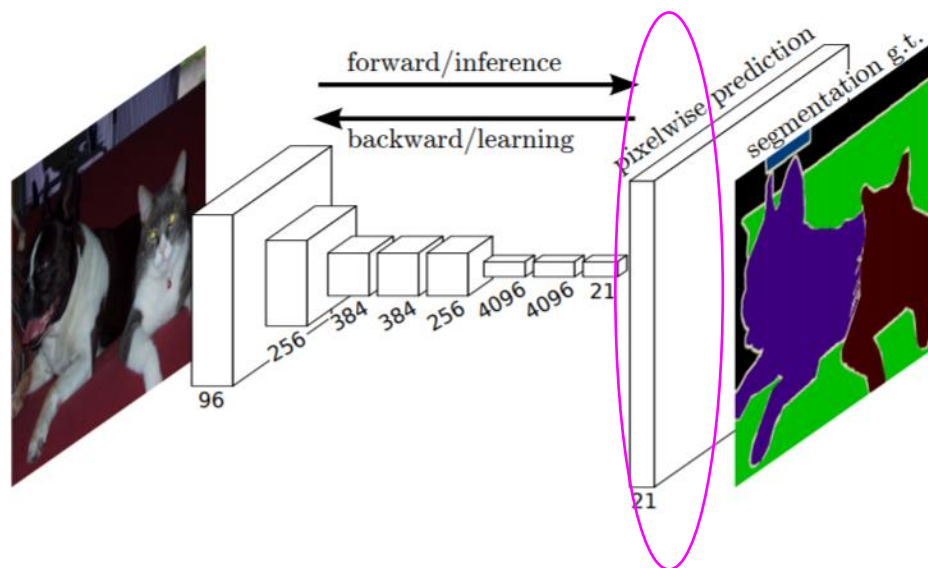


缩小后的图像1.5倍放大
最近邻插值

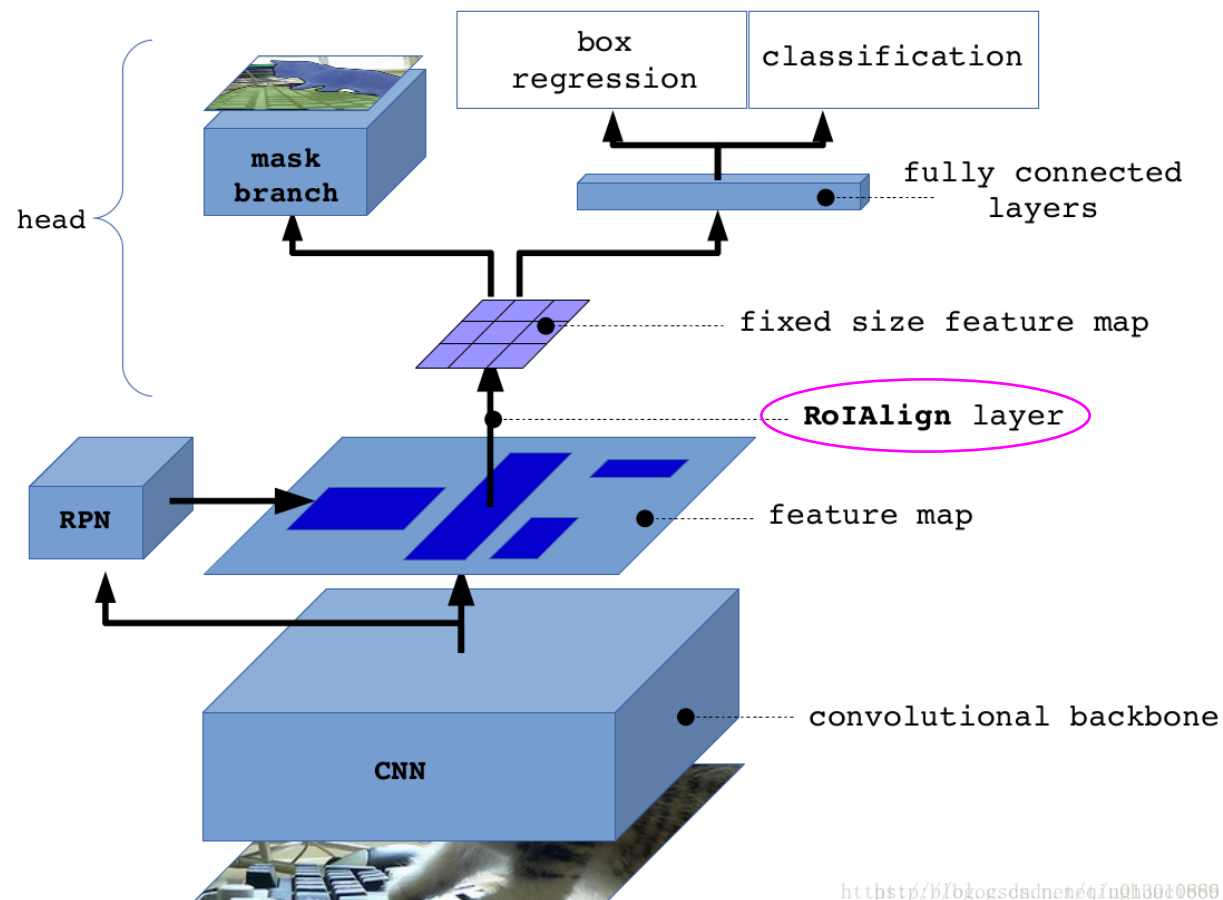


缩小后的图像1.5倍放大
双线性插值

图像插值在深度学习领域的应用举例



Fully Convolutional Network (FCN)



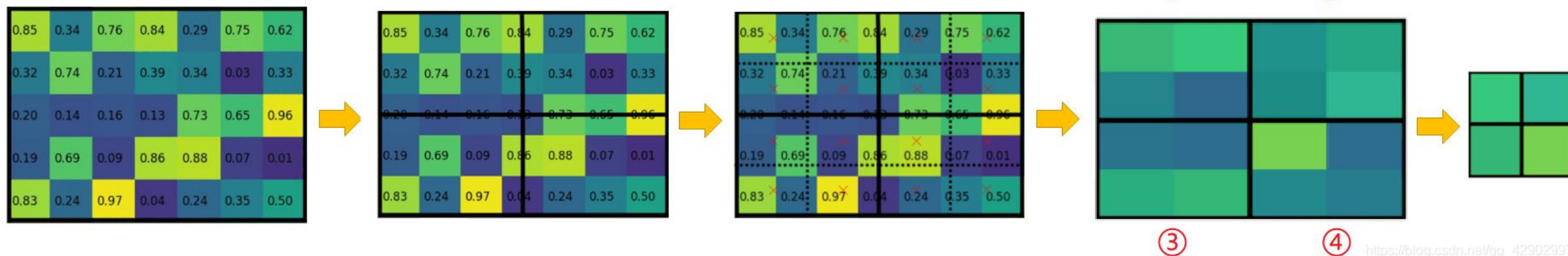
Mask R-CNN

<http://b1b0g0s0d0n0e0q0i0n0h0u0t00660>

多分辨率表达

扩展阅读：Mask RCNN中的ROI Align

https://blog.csdn.net/qq_42902997/article/details/105087407



https://blog.csdn.net/qq_42902997



图像几何变换

详见实践课PPT：第四章，第9-19页

扩展阅读：<https://mp.weixin.qq.com/s/wvEv4C97QRAtXQ3TISToEw>

图像的旋转变换在深度学习领域的应用举例



数据扩充 (data augmentation)

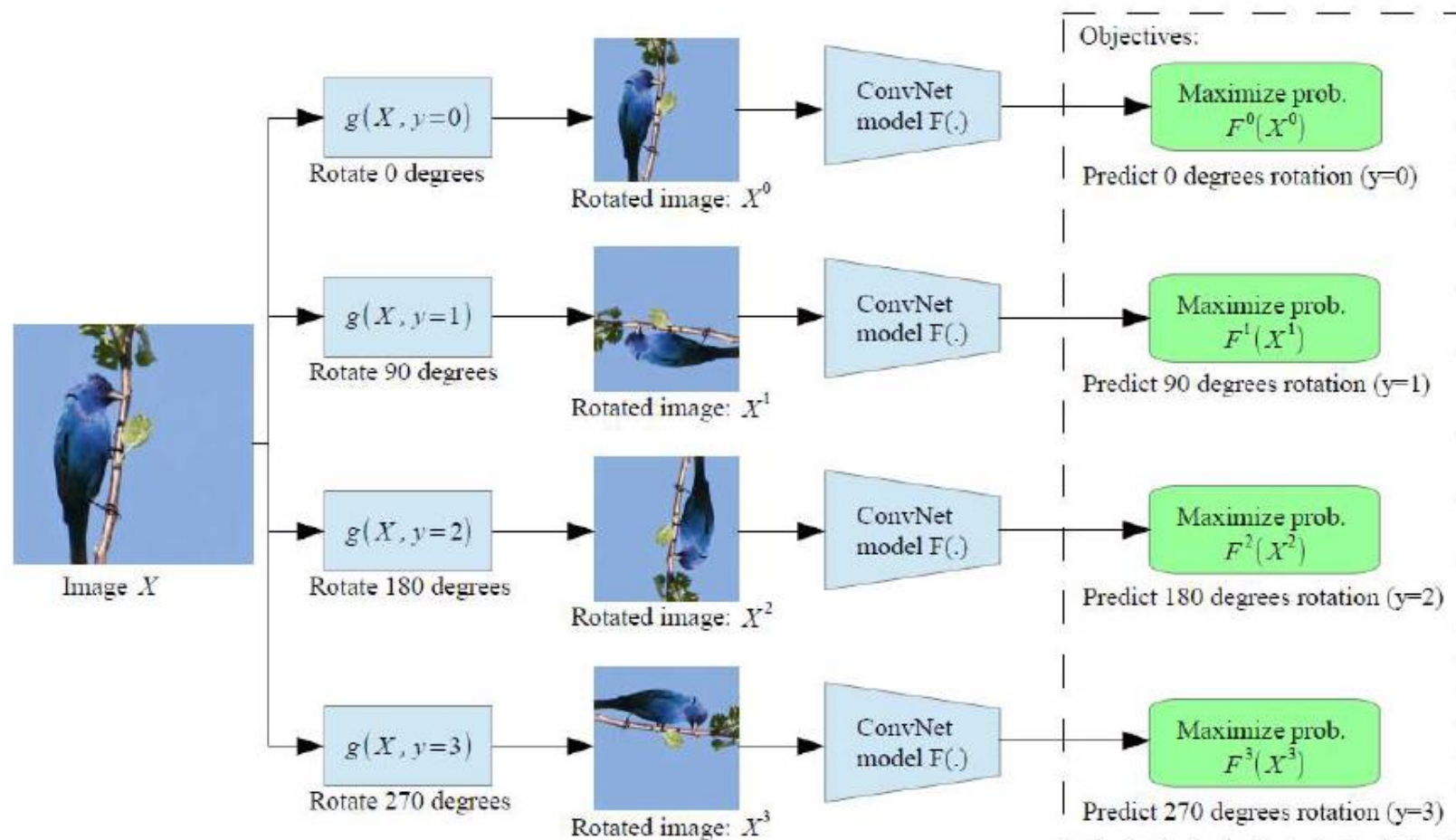
Which image has the correct rotation?



Unsupervised representation learning by predicting image rotations,
Spyros Gidaris, Praveer Singh, Nikos Komodakis, ICLR 2018

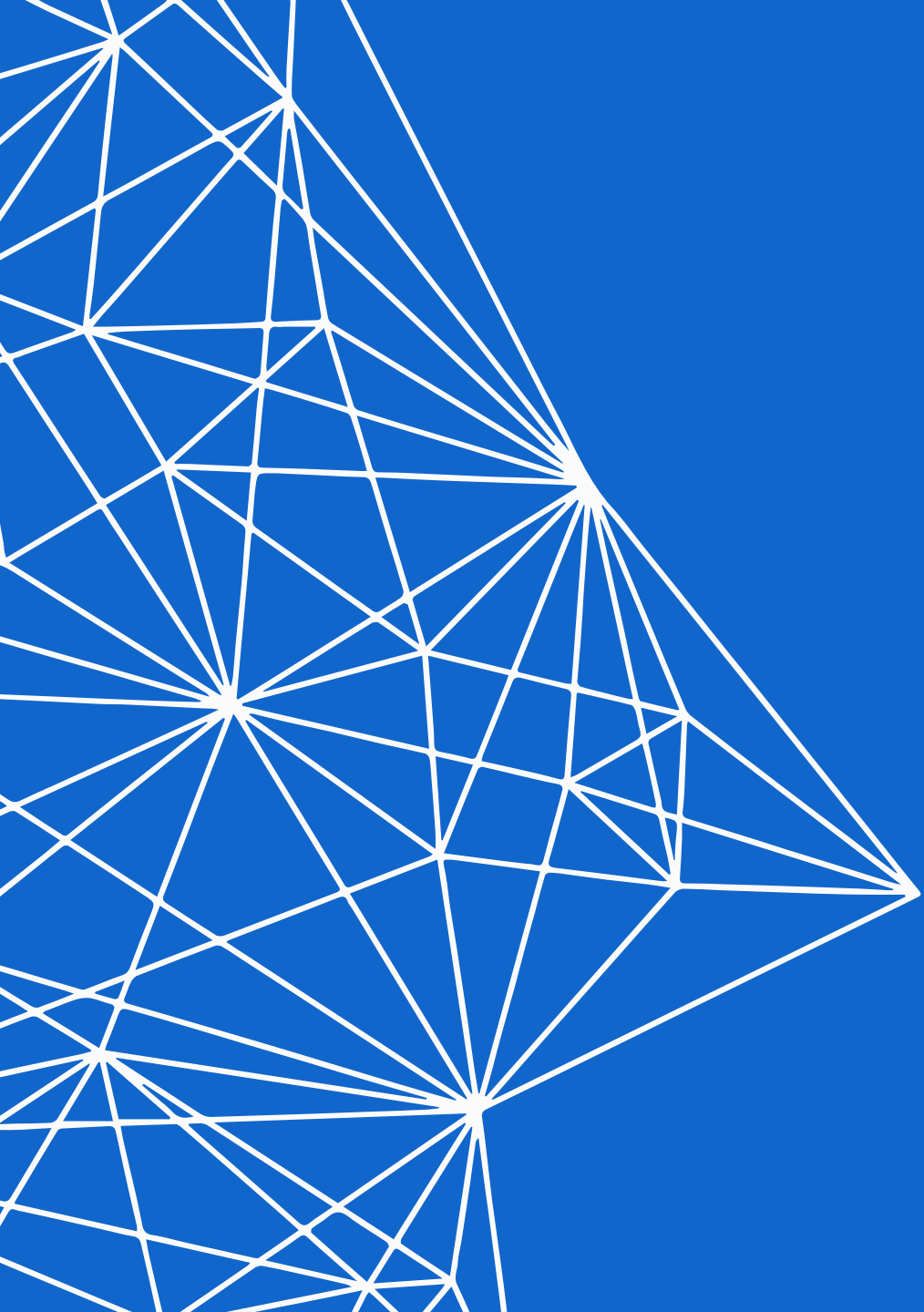
自监督学习 (Self-supervised learning)

图像的旋转变换在深度学习领域的应用举例



Unsupervised representation learning by predicting image rotations,
Spyros Gidaris, Praveer Singh, Nikos Komodakis, ICLR 2018

自监督学习 (Self-supervised learning)



THANKS!